Lab 05: Complexity Hell

Zakamarki złożoności cyklomatycznej

Jarosław Hryszko
jaroslaw.hryszko@uj.edu.pl

Instytut Informatyki i Matematyki Komputerowej UJ

Otwarte repozytoria kodu i pomiary oprogramowania

Złożoność cyklomatyczna – intuicja

Pytanie

Która funkcja jest trudniejsza do zrozumienia?

Funkcja A (CC = 1)

```
def add(a, b):
    return a + b
```

Jedna ścieżka przez kod.

Funkcja B (CC = 4)

```
def classify(x):
    if x > 0:
        if x > 100:
            return "big"
        return "positive"
    elif x == 0:
        return "zero"
    return "negative"
```

Cztery niezależne ścieżki.

Jak liczyć CC?

Wzór McCabe'a (1976)

$$CC = E - N + 2P$$

E = krawędzie grafu, N = węzły, P = komponenty

Prościej: zaczynamy od 1 i dodajemy 1 za każdy:

- `if`, `elif`
- `for`, `while`
- `and`, `or` (w warunkach)
- `except`
- `assert`
- `with` (kontekstowo)

Każdy z tych operatorów tworzy nową ścieżkę w kodzie – czyli nowy scenariusz do przetestowania.

Interpretacja wyników

CC	Ranking	Co to znaczy?
1–5	A	Prosta, łatwa w testowaniu
6–10	B	Umiarkowana, do ogarnięcia
11–20	C	Złożona, kandydat do refaktoryzacji
21–50	D	Bardzo złożona, trudna w utrzymaniu
50+	F	Nie do utrzymania

CC mówi nam też **ile minimum testów** potrzeba, żeby pokryć wszystkie ścieżki.

Narzędzia

radon (tylko Python)

- `radon cc` – złożoność cyklomatyczna
- `radon mi` – indeks utrzymywalności: jedna liczba mówiąca, jak łatwo zrozumieć i zmienić kod
- `radon hal` – metryki Halsteada: szacują wysiłek programisty na podstawie operatorów i operandów
- Output: tekst lub JSON (`-j`)

lizard (wielojęzyczny)

- C/C++, Java, Python, JS, Go...
- CC + NLOC + token count
- Konfigurowane progi
- Output: tekst lub CSV

Dziś: porównamy oba, a potem napiszemy **własny profiler** oparty na radonie.

Plan zajęć

1. **Zadanie 1 – radon i lizard** (30 min)
   - Analiza projektu OSS oboma narzędziami
   - Porównanie wyników, znalezienie „najgorszej” funkcji
2. **Zadanie 2 – Complexity Profiler** (60 min)
   - Skrypt `complexity_profiler.py`
   - Top 20, statystyki, rozkład A/B/C/D/F, histogram
3. **Zadanie 3 – CC vs bugi** (45 min, opcjonalne)
   - Korelacja złożoności z liczbą bugfix commitów
   - Scatter plot

Co oddajecie?

W branchu `lab05_nazwisko1_nazwisko2`:

1. `complexity_profiler.py` – działający skrypt
2. `complexity_histogram.png` – histogram
3. `answers.md` – porównanie radon vs lizard, analiza funkcji
4. *(opcjonalnie)* `complexity_vs_bugs.png`

Do roboty!

Instrukcja: `README.md` w repozytorium

"Prostota jest warunkiem koniecznym niezawodności." – Edsger Dijkstra